```
.../validations.rb:736:in `save!': Validation failed: Balance is negative
from transactions.rb:46:in `adjust_balance_and_save!'
  :        :        :
from transactions.rb:80
```

Looking in the database, we can see that the data remains unchanged:

```
depot> sqlite3 -line storage/development.sqlite3 "select * from accounts"
     id = 1
 number = 12345
balance = 100

     id = 2
 number = 54321
balance = 200
```

However, there's a trap waiting for you here. The transaction protected the database from becoming inconsistent, but what about our model objects? To see what happened to them, we have to arrange to intercept the exception to allow the program to continue running:

rails80/e1/ar/transactions.rb

```
peter = Account.create(balance: 100, number: "12345")
paul  = Account.create(balance: 200, number: "54321")
```

rails80/e1/ar/transactions.rb

```
begin
  Account.transaction do
    paul.deposit(350)
    peter.withdraw(350)
  end
rescue
  puts "Transfer aborted"
end

puts "Paul has #{paul.balance}"
puts "Peter has #{peter.balance}"
```

What we see is a little surprising: